INFORME TÉCNICO · IMPLEMENTACIÓN NoSQL Y MANIPULACIÓN DE DATOS

Caso ContentFlow (Forma A) · Evaluación Parcial N°2 (40%)

Institución	Duoc UC · Sede Puerto Montt · Escuela de Informática y Telecomunicaciones
Asignatura	BDY0101 · Base de Datos No Relacionales
Integrantes	Carlos Ignacio Bittner Navea · Vicente Benjamin Alarcón Gallardo · Francisco Jesús Aranguiz Inostroza
Tecnología	Azure Cosmos DB for MongoDB (Serverless, modelo RU) · Python 3.12 · PyMongo 4.x · FastAPI · Postman
Datos	5 colecciones · 2.500 documentos (≈ 0,66 MB en JSON)
Repositorio	https://github.com/ChenteDev1/ContentFlow
Versión	2.0 · Final

---

1. Objetivos del proyecto

ContentFlow es una plataforma donde las personas publican contenido, reaccionan, comentan y participan en campañas. La gerencia quiere saber **quiénes son los usuarios más activos, qué contenidos son más populares y qué impacto tienen las campañas**, para decidir sobre diseño de contenido, horarios de publicación, perfil de audiencia y éxito de campañas.

Objetivo general. Implementar sobre Azure Cosmos DB for MongoDB una base de datos NoSQL para ContentFlow, consultarla y manipularla de forma segura y medible, e integrarla con una aplicación externa (API REST) con trazabilidad y control de errores.

Objetivos específicos (entre paréntesis, los indicadores de la pauta que evidencian cada uno):

1. Configurar la cuenta Cosmos DB (API, capacidad, consistencia, partición, throughput, credenciales) y verificar su conectividad con varias herramientas (IE1, IE2, IE3).
2. Cargar y verificar los 2.500 documentos de prueba de forma repetible (IE1, IE3, IE7).
3. Construir consultas de lectura y modificación con filtros, proyecciones, búsquedas compuestas y ordenamientos que respondan a requerimientos de negocio, y evaluar su eficiencia con tiempo, volumen y costo en RU (IE4, IE5, IE6).
4. Implementar un CRUD completo sobre varias colecciones que simule un caso real, validando duplicados, inconsistencias y pérdida de contexto (IE7, IE8).
5. Integrar la base con una API REST con lectura y escritura en tiempo real, funciones de negocio, registros de actividad y control de errores (IE9, IE10, IE11).

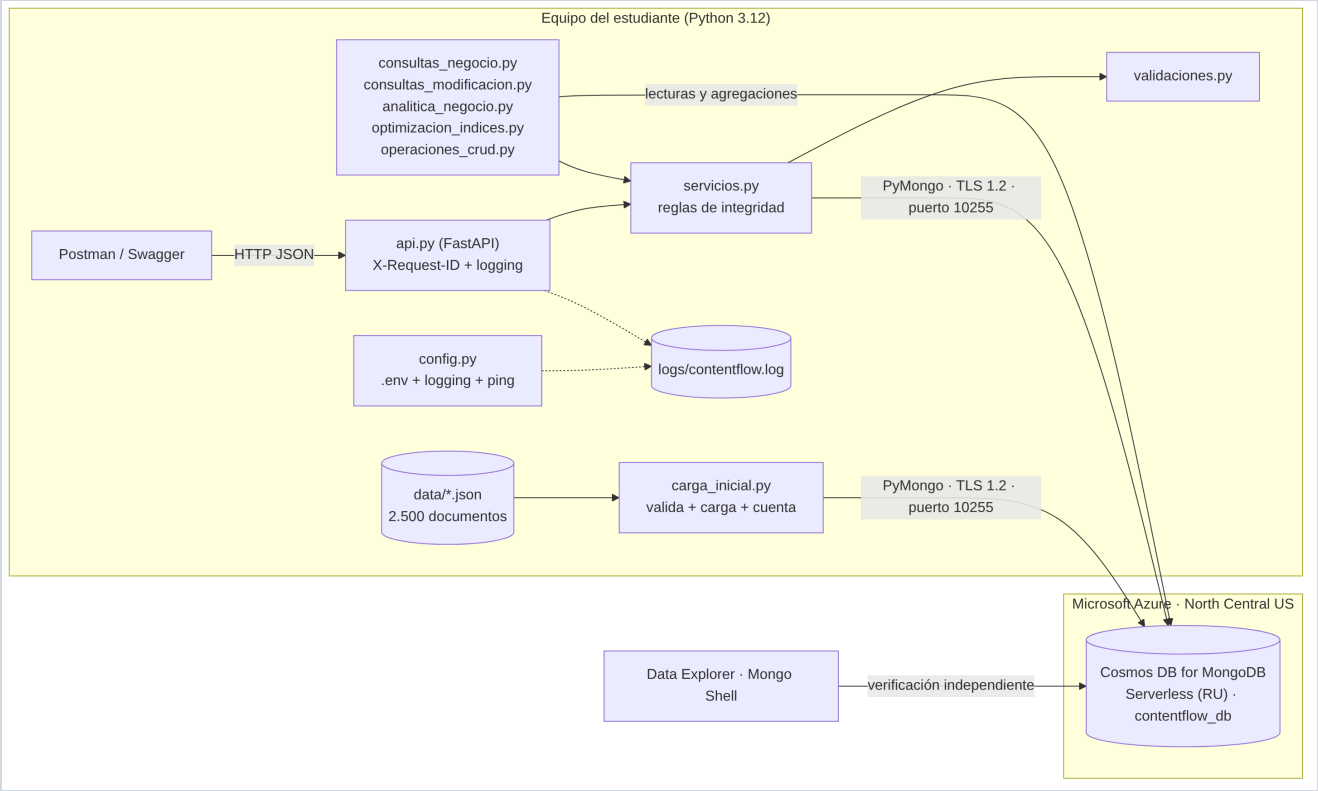


Figura 1. Arquitectura de la solución

2. Configuración de la base de datos NoSQL

2.1 API seleccionada y justificación

Se eligió **Azure Cosmos DB for MongoDB** (la pauta la permite como alternativa a MongoDB Atlas) en lugar de las otras APIs de Cosmos DB, por estas razones:

API de Cosmos DB	Modelo	¿Calza con ContentFlow?
for MongoDB (elegida)	Documentos JSON/BSON con arrays y objetos anidados	**Sí.** El caso describe una base MongoDB; los datos ya son JSON (hashtags como arreglo, `metricas_objetivo` como subdocumento); se usan operadores conocidos (`$in`, `$gte`, `$group`, `$inc`, `$addToSet`); se accede con PyMongo, Mongo Shell y Data Explorer.
NoSQL (Core/SQL)	Documentos con lenguaje SQL propio y SDK propio	Posible, pero obliga a reescribir consultas y cambia el ecosistema respecto del caso.
Gremlin	Grafos (vértices y aristas)	Útil si el foco fueran recorridos de relaciones profundas (“amigos de amigos”). Aquí las relaciones son simples (1 usuario → N publicaciones) y se resuelven con referencias.
Table	Clave–valor	Sin filtros compuestos, proyecciones ni agregaciones: insuficiente para las consultas pedidas.

Modelo documental frente al relacional. Una publicación guarda sus `hashtags` embebidos en el mismo documento (en SQL requeriría tablas puente), el esquema puede evolucionar sin `ALTER TABLE` y las lecturas por `_id` no requieren JOIN. A cambio, **no existen llaves foráneas ni integridad referencial automática**: esa responsabilidad pasa a la aplicación (secciones 5 y 7).

2.2 Recurso creado en Azure

Datos tomados de la captura de creación (Figura 2):

Parámetro	Valor
Suscripción / grupo de recursos	Azure for Students / `ContentFlowVA`
Ubicación	North Central US
Cuenta	`cosmos-contentflow-mongo-va`
API	Azure Cosmos DB for MongoDB (cuenta RU)
Modo de capacidad	**Sin servidor (Serverless)**
Zonas de disponibilidad	Habilitadas
Copia de seguridad	Periódica, almacenamiento con redundancia geográfica
Cifrado	En reposo, administrado por el servicio · TLS mínimo 1.2
Conectividad	"Todas las redes" (adecuado para el desarrollo; en producción se restringiría por IP o red virtual)
Base de datos	`contentflow_db`
Endpoint	`cosmos-contentflow-mongo-va.mongo.cosmos.azure.com:10255`

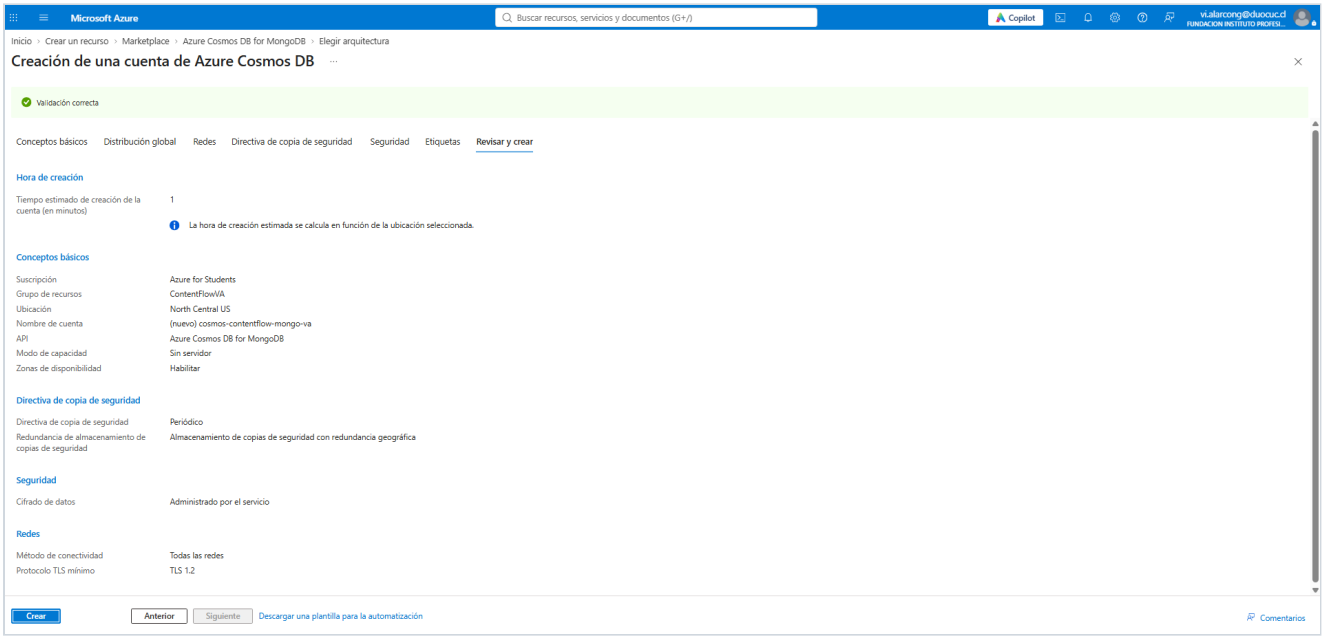


Figura 2. Revisión y creación de la cuenta en Azure Portal

2.3 Estructura de colecciones ("contenedores")

Se crearon 5 colecciones con `_id` de texto (`user_1`, `pub_1`, `react_1`, `comm_1`, `camp_1`). Las colecciones que crecen sin límite (`reacciones`, `comentarios`) **referencian** a su usuario y publicación por id en vez de incrustarse; en cambio lo que pertenece a un único documento y se lee junto con él se **incrusta** (`hashtags[]`, `metricas_objetivo{}`).

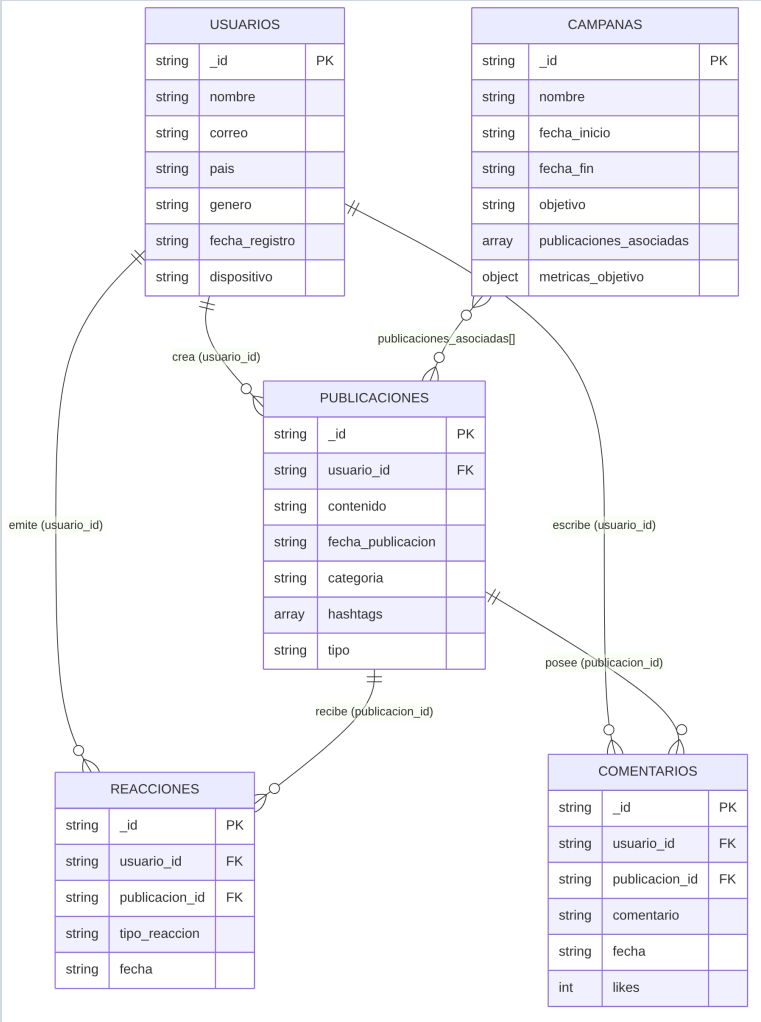


Figura 3. Modelo de datos (referencias por id)

2.4 Claves de partición

- **Estado actual.** Las colecciones se crearon al insertar los datos, **sin clave de partición** (colecciones no particionadas). Con ≈ 0,66 MB y 2.500 documentos esto es suficiente y mantiene simples las operaciones por `_id`.
- **Propuesta para crecer.** Una clave de partición debe tener alta cardinalidad y aparecer en las consultas más frecuentes:

Colección	Clave propuesta	Motivo
`usuarios`	`_id`	Alta cardinalidad; acceso puntual por usuario.
`publicaciones`	`usuario_id`	"Publicaciones de un usuario" es una consulta frecuente.
`reacciones`, `comentarios`	`publicacion_id`	Se leen por publicación (feed). Riesgo: una publicación viral concentra datos en una partición.
`campañas`	`_id`	Pocas escrituras; lectura por campaña.

- **Cómo se aplica.** `python carga_inicial.py --shard-key` recrea las colecciones con `shardCollection` (clave *hashed*). Es opcional porque la clave solo puede definirse al crear la colección y las operaciones sobre colecciones particionadas deben volver a probarse; **en esta entrega las colecciones no están particionadas** y la tabla anterior es la propuesta de diseño.

2.5 Throughput

En **Serverless no se aprovisionan RU/s**: se factura por las Request Units que consume cada operación y la cuenta admite hasta 5.000 RU/s de ráfaga por contenedor. Es adecuado para esta carga (desarrollo, lotes y picos aislados) porque no hay costo en reposo ni que dimensionar capacidad, y se evita el error 429 por subdimensionar. Para medir el consumo real, `metricas.py` ejecuta el comando propio de Cosmos `getLastRequestStatistics` tras cada consulta (columna **RU** de la sección 4).

2.6 Consistencia

Se usa el nivel por defecto de Cosmos DB, **Session**: dentro de una misma sesión de cliente se garantiza leer lo que se acaba de escribir (*read-your-writes*), que es lo que ocurre en los flujos "crear y luego consultar" del CRUD y de la API, con menor costo de RU y latencia que *Strong*.

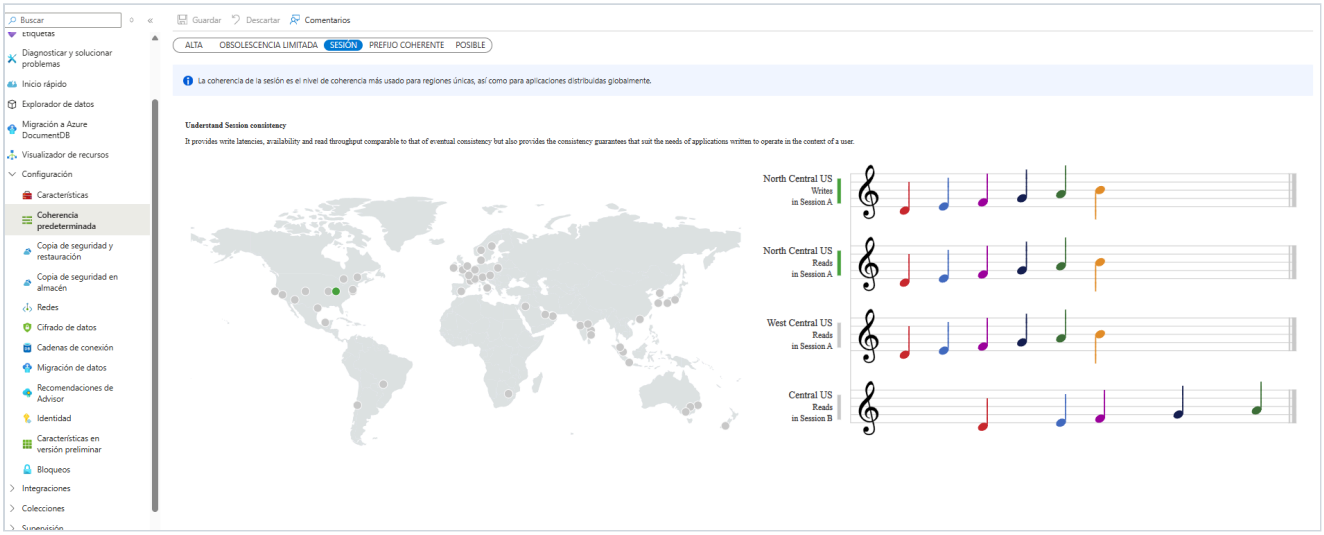


Figura 4. Coherencia predeterminada de la cuenta

2.7 Credenciales y seguridad

- La cadena de conexión vive solo en `.env` (excluido por `.gitignore`); el repositorio incluye `.env.example` con `<PRIMARY_KEY>`.
- El código **no tiene cadena por defecto** y se detiene con un mensaje claro si falta o conserva `<PRIMARY_KEY>`.
- El log enmascara credenciales (`mongodb://***:***@…`) y la API nunca devuelve detalles internos de la base al cliente.
- Se revisaron las capturas del repositorio: ninguna muestra la clave ni la cadena de conexión. Recomendación: **rotar la clave** si alguna vez se pegó en un chat o commit.

2.8 Problemas encontrados y resueltos

Problema	Causa	Solución
Error de escritura con la cadena genérica de MongoDB	Cosmos DB no admite *retryable writes*	`retrywrites=false` en la cadena de conexión.
Conexiones que se cuelgan sin respuesta	Timeout por defecto de 30 s	`serverSelectionTimeoutMS=5000` + `ping` inicial → falla en 5 s con mensaje claro (`ConexionError`).

Problema	Causa	Solución
Consulta 3 original devolvía las 500 campañas y decía "Presupuesto ≥ $5.000"	Buscaba un campo `presupuesto` que **no existe** en el dataset	Se reescribió con campos reales (`objetivo`, `metricas_objetivo.likes`, `fecha_inicio`).
Ordenar por un campo sin índice	Cosmos exige índice para ordenar en el servidor	Se intenta ordenar en el servidor y, si lo rechaza, se ordena en memoria y se registra; se miden los índices (sección 4.4).
La carga borraba la colección *antes* de comprobar el archivo	Orden de operaciones en `carga_inicial.py`	Ahora se validan los 5 JSON primero y luego se carga; se cuenta lo insertado.
Fechas nuevas con formato distinto al dataset (`…+00:00` con microsegundos)	`isoformat()` en el CRUD original	Formato único `YYYY-MM-DDTHH:MM:SS`, igual al de los JSON (si no, los textos no ordenan bien).
Advertencia de PyMongo "connected to a CosmosDB cluster"	Aviso informativo del driver	Se filtra y se documenta; no afecta el funcionamiento.
Cadena de conexión copiada en 3 scripts	Código duplicado	Un solo módulo `config.py`.

---

3. Inserción de datos de prueba

3.1 Dataset

Colección	Documentos	Campos
`usuarios`	500	nombre, correo, país, género, fecha_registro, dispositivo (Desktop/Móvil/Tablet)
`publicaciones`	500	usuario_id, contenido, fecha_publicacion, categoría (5), hashtags[1–3], tipo (Video/Texto/Imagen)
`reacciones`	500	usuario_id, publicacion_id, tipo_reaccion (4), fecha
`comentarios`	500	usuario_id, publicacion_id, comentario, fecha, likes (0–100)
`campañas`	500	nombre, fecha_inicio/fin, objetivo (3), publicaciones_asociadas[1–5], metricas_objetivo{likes, comentarios, compartidos}

3.2 Carga repetible (`carga_inicial.py`)

1. **Valida** los 5 archivos (lista no vacía, objetos con `_id`, sin `_id` repetidos) *antes* de tocar la base.
2. Conecta con timeout y `ping`.
3. Por colección: `delete_many({})` (estado inicial conocido) → `insert_many` → `count_documents` y compara con el archivo.
4. Registra todo en `logs/contentflow.log` y termina con código distinto de 0 si algún conteo no coincide.

`python carga_inicial.py --solo-validar` permite revisar los archivos sin conectarse.

3.3 Evidencias de inserción y verificación

Se verificó por **cuatro vías** distintas (consola Python, Data Explorer, Mongo Shell y la API `GET /salud`):

```
PS C:\Users\alarc\OneDrive\Escritorio\ContentFlow> python carga_inicial.py
Leyendo archivo: data\reacciones.json...
-> Insertados 500 registros en 'reacciones'.
Leyendo archivo: data\comentarios.json...
-> Insertados 500 registros en 'comentarios'.
Leyendo archivo: data\campañas.json...
-> Insertados 500 registros en 'campañas'.

¡Carga inicial completada con éxito!
PS C:\Users\alarc\OneDrive\Escritorio\ContentFlow>
```

Figura 5. Ejecución de carga_inicial.py (consola)

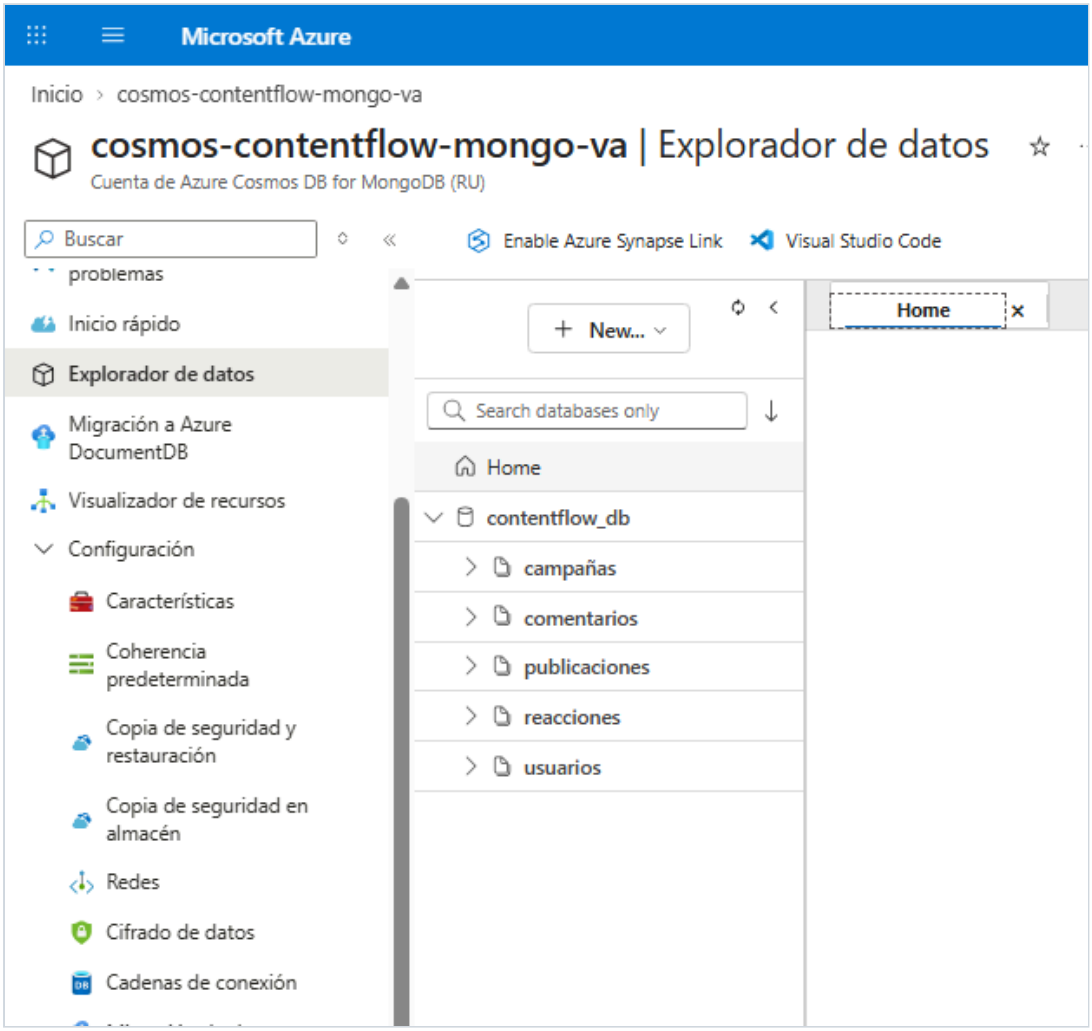


Figura 6. Las 5 colecciones creadas en Data Explorer



Figura 7. 500 documentos seleccionados en `campañas` (Data Explorer)



Figura 8. `countDocuments()` de cada colección en Mongo Shell

3.4 Conectividad con herramientas externas (IE3)

Herramienta	Qué se comprobó	Evidencia
Data Explorer (Azure Portal)	Colecciones y 500 documentos por colección	Figuras 6 y 7
Mongo Shell (extensión de VS Code)	Conteos con `countDocuments()`	Figura 8
SDK PyMongo (scripts Python)	Carga, consultas y CRUD	Figuras 5, 9–13
Postman / Swagger vía API REST	`GET /salud` (ping) y flujo CRUD	Sección 6

3.5 Calidad del dataset (hallazgos de `integridad.py`)

Las referencias entre colecciones son **íntegras** (0 huérfanos). Se encontraron, en cambio, problemas de calidad que se documentan y se tratan en la sección 5.3:

Hallazgo	Cantidad
Publicaciones, reacciones o comentarios con usuario o publicación inexistente	0
Campañas con publicaciones inexistentes o con fecha de fin anterior al inicio	0
Correos de usuario repetidos	0
Reacciones duplicadas (mismo usuario y misma publicación)	**1**
Publicaciones anteriores al registro de su autor	48
Reacciones anteriores a la publicación reaccionada	245

Hallazgo	Cantidad
Comentarios anteriores a la publicación comentada	251
Reacciones / comentarios anteriores al registro de su autor	53 / 54

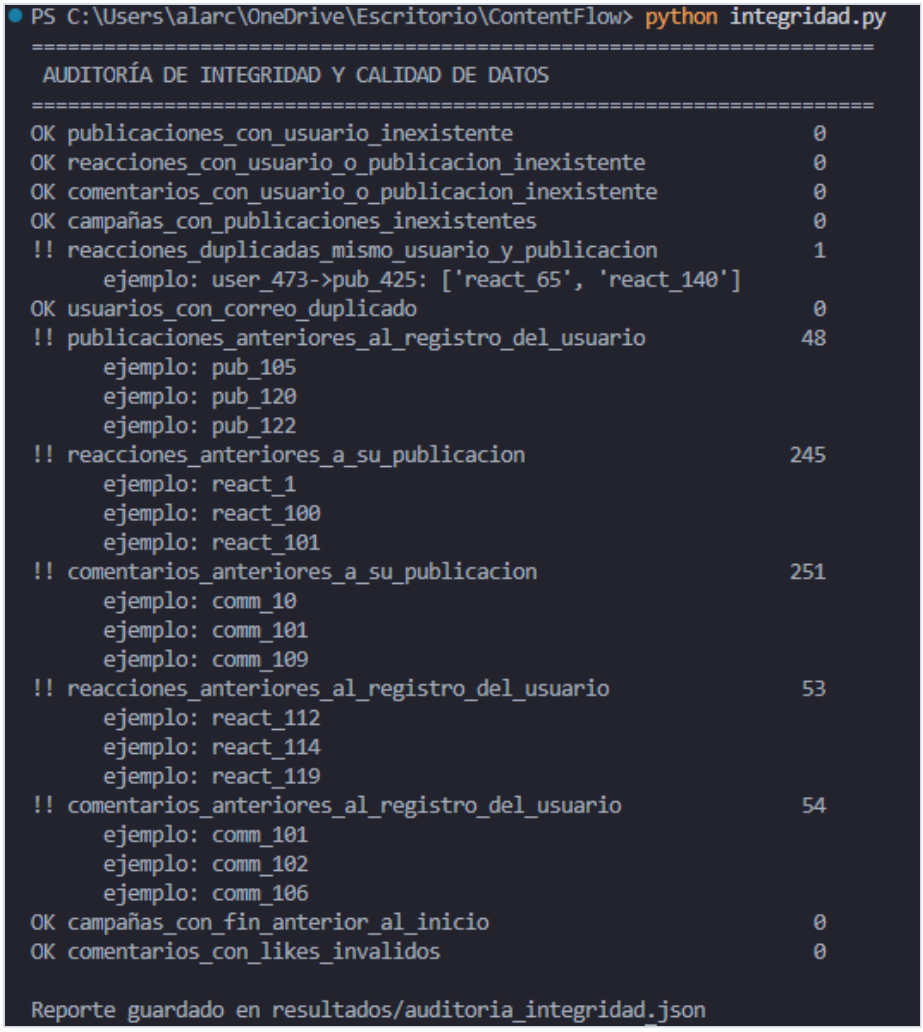


Figura 9. Salida de `python integridad.py` contra Azure

4. Consultas sobre la base de datos

4.1 Consultas de lectura (`consultas_negocio.py`)

Cada consulta responde una necesidad de negocio y usa filtro, proyección y orden pedidos al servidor:

N°	Requerimiento de negocio	Colección	Filtro	Proyección	Orden
1	Segmento Desktop masculino para diseño web y patrocinios	`usuarios`	`dispositivo = Desktop` **y** `genero = Masculino`	nombre, correo, país	nombre ↑
2	Auditoría de cobertura de Deportes	`publicaciones`	`categoria = Deportes`	usuario, tipo, fecha, hashtags	fecha ↓

N°	Requerimiento de negocio	Colección	Filtro	Proyección	Orden
3	Campañas de alto impacto	`campañas`	`objetivo ∈ {Aumentar engagement, Promover contenido}` **y** `metricas_objetivo.likes ≥ 500` **y** `fecha_inicio ≥ 2025-02-01`	nombre, objetivo, fechas, métricas	likes meta ↓
4	Selección de videos/imágenes de tendencia	`publicaciones`	`tipo ∈ {Video, Imagen}` **y** `hashtags ∈ {#cultura, #series}` **y** `fecha ∈ [2025-06-01, 2025-08-01)`	categoría, tipo, hashtags, fecha	fecha ↓
5	Comentarios a destacar	`comentarios`	`likes ≥ 90`	usuario, publicación, texto, likes	likes ↓, top 10

Operadores utilizados: `$in`, `$gte`, `$lt`, filtro compuesto (AND implícito), notación con punto sobre subdocumento (`metricas_objetivo.likes`), búsqueda dentro de arreglo (`hashtags`), proyección, `sort` y `limit`.

```
# consultas_negocio.py · Consulta 3 (extracto)
filtro = {
    "objetivo": {"$in": ["Aumentar engagement", "Promover contenido"]},
    "metricas_objetivo.likes": {"$gte": 500},
    "fecha_inicio": {"$gte": "2025-02-01T00:00:00"},
}
db.campañas.find(filtro, {"_id": 0, "nombre": 1, "objetivo": 1, "metricas_objetivo": 1}) \
           .sort([("metricas_objetivo.likes", -1)])
```

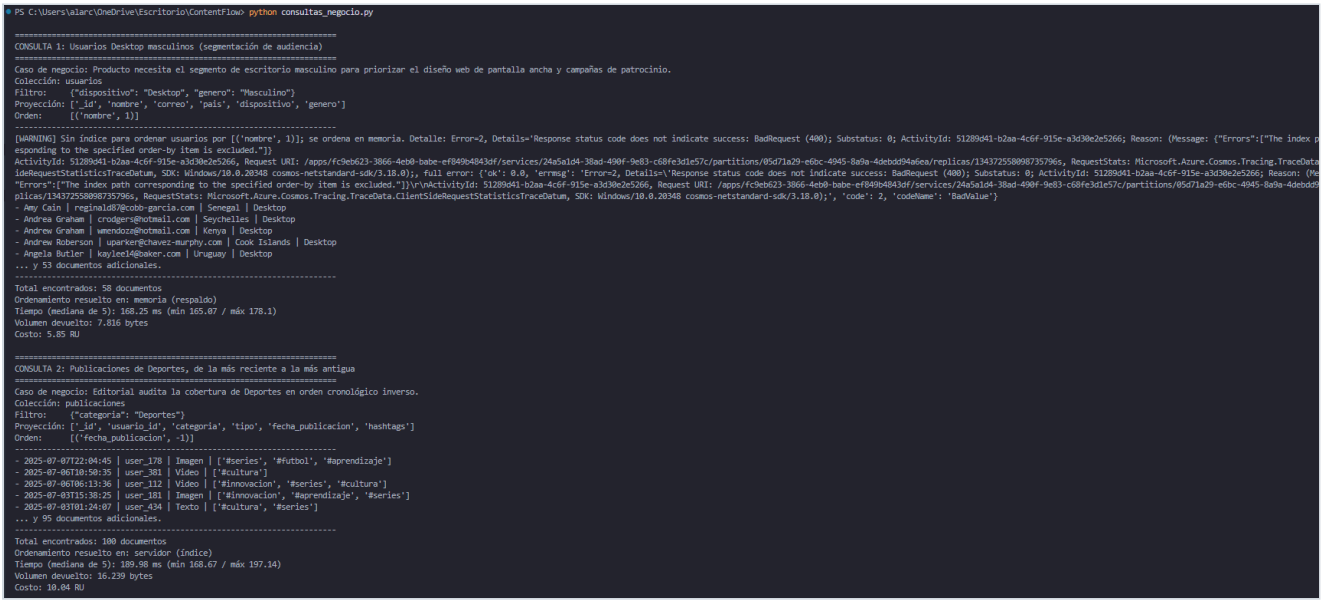


Figura 10. Ejecución de `python consultas_negocio.py` contra Azure

Resultados medidos en Azure

Se mide la **mediana de 5 ejecuciones** (tras una de calentamiento), los **bytes BSON** devueltos y las **RU** del comando `getLastRequestStatistics`.

N°	Consulta	Docs	Orden resuelto en	Mediana ms (mín–máx)	Bytes	RU
1	Usuarios Desktop masculinos (segmentación de audiencia)	58	memoria (respaldo)	168.25 (165.07–178.1)	7.816	5.85
2	Publicaciones de Deportes, de la más reciente a la más antigua	100	servidor (índice)	189.98 (168.67–197.14)	16.239	10.04
3	Campañas de alto impacto (objetivo + métricas + fecha)	56	servidor (índice)	193.82 (189.01–209.82)	11.650	18.8
4	Videos recientes con hashtags de tendencia (búsqueda compuesta)	67	servidor (índice)	191.63 (188.35–193.38)	10.820	14.92
5	Comentarios más valorados (Top 10 por likes)	63	memoria (respaldo)	189.95 (186.33–195.48)	8.792	14.69

Línea base de la primera versión (una sola ejecución, de las capturas originales del repositorio): Consulta 1 = 192,61 ms (58 docs); Consulta 2 = 234,68 ms (100 docs); Consulta 3 anterior = 646,18 ms (500 docs, **sin filtro real**). Las cifras que figuraban en el informe anterior (420,19 / 374,38 / 621,09 ms) **no coincidían** con esas capturas y se descartan.

4.2 Consultas de modificación (`consultas_modificacion.py`)

Cada una cuenta lo que va a tocar, aplica el cambio, **verifica** y por defecto **revierte** para no alterar el dataset.

N°	Operación	Operador	Verificación
M1	Marcar como “Destacada” las campañas de engagement con meta ≥ 900 likes (18 documentos)	`update_many` + `$set`	`matched = modified = verificadas`
M2	Agregar `#VeranoDeportivo` a videos de Deportes desde el 15-jun-2025 (5 documentos)	`update_many` + `$addToSet`	Una segunda ejecución modifica **0** (idempotente)
M3	Un like más a un comentario	`update_one` + `$inc`	`likes` aumenta exactamente 1 (atómico, no pisa otros likes)
M4	Cerrar una campaña: estado, fecha de fin y +25 compartidos	`update_one` + `$set` + `$inc` anidado	Se compara con la copia original y se restaura con `replace_one`

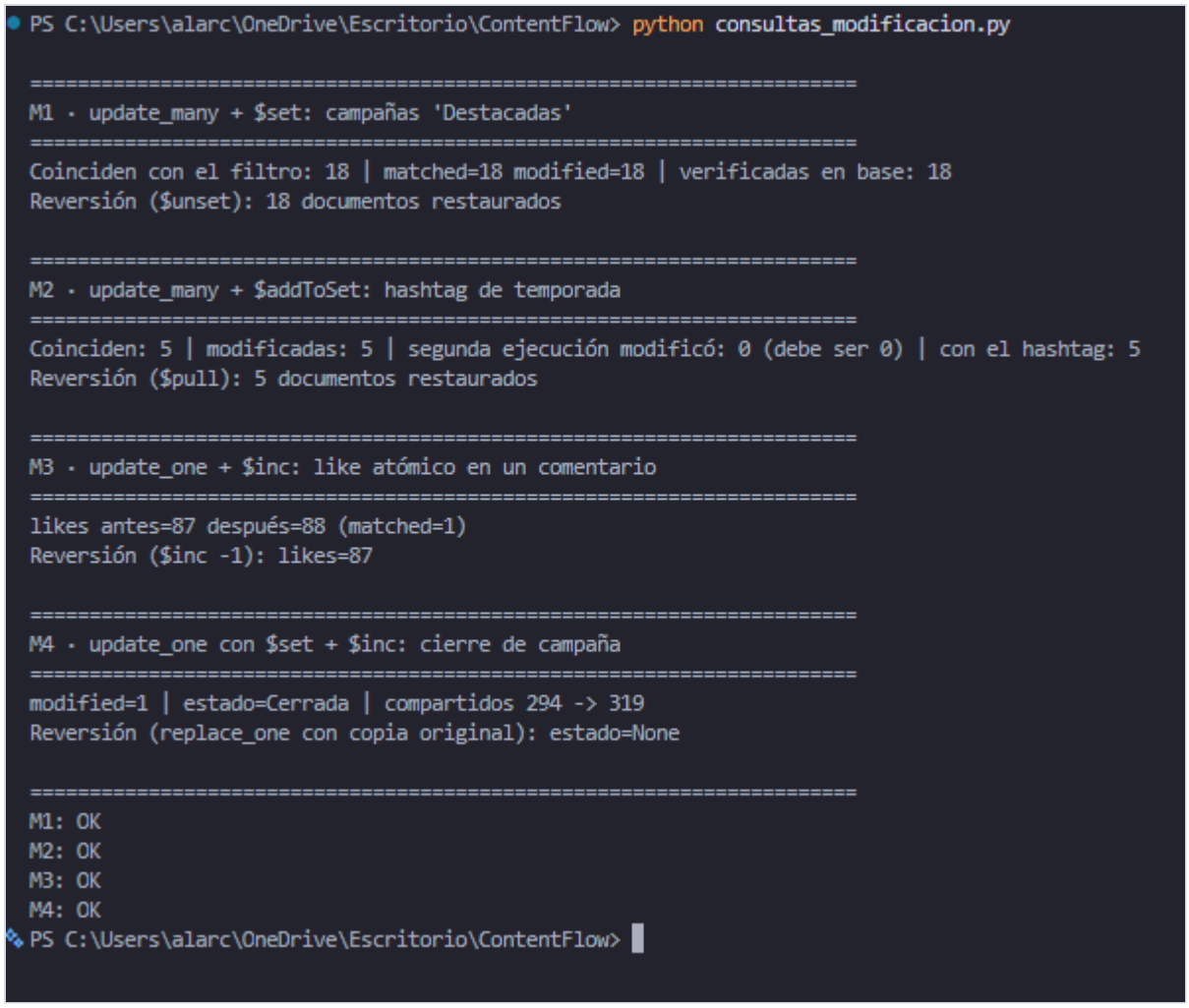


Figura 11. Ejecución de `python consultas_modificacion.py` contra Azure

4.3 Analítica para la gerencia (`analitica_negocio.py`)

Responde las tres preguntas del caso con agregaciones del servidor (`$group`, `$sum`, `$avg`, `$substr`, `$sort`); el cruce entre colecciones se hace en la aplicación con consultas `$in` (más simple y predecible en RU que `$lookup`). Los resultados siguientes se calcularon con los datos reales del caso.

a) Usuarios más activos. Índice de actividad = 3·publicaciones + 2·comentarios + 1·reacciones (ponderación definida por el equipo: crear contenido aporta más que reaccionar).

Usuario	Nombre	País	Publ.	Coment.	Reacc.	Índice
user_205	Kathleen Jones	Fiji	4	3	2	20
user_73	Felicia Jensen	Cook Islands	4	2	3	19
user_359	Misty Meyer	Saudi Arabia	4	2	2	18
user_382	William Giles	Maldives	5	1	1	18
user_206	Matthew Hatfield	Timor-Leste	3	4	0	17
user_311	Michael Bass	Botswana	3	1	6	17
user_353	Wendy Carpenter	Iraq	4	2	1	17
user_292	Barbara Wood	Kenya	3	3	1	16
user_300	Tammy Weeks	Sierra Leone	2	4	2	16

Usuario	Nombre	País	Publ.	Coment.	Reacc.	Índice
user_391	Michael Ware	Poland	3	3	1	16

b) Contenidos más populares (reacciones + comentarios) y popularidad por categoría.

Publicación	Categoría	Tipo	Reacc.	Coment.	Likes en coment.	Interacciones
pub_237	Deportes	Imagen	3	5	385	8
pub_312	Educación	Video	4	3	249	7
pub_50	Cultura	Video	4	3	119	7
pub_381	Deportes	Texto	1	5	284	6
pub_134	Tecnología	Texto	1	5	229	6
pub_425	Cultura	Texto	3	3	171	6
pub_41	Tecnología	Imagen	3	3	142	6
pub_393	Entretenimiento	Video	3	3	120	6
pub_358	Cultura	Imagen	4	2	94	6
pub_388	Cultura	Video	4	2	82	6

Categoría	Publicaciones	Interacciones	Promedio por publicación
Entretenimiento	82	174	2.12
Educación	102	214	2.1
Tecnología	111	220	1.98
Deportes	100	195	1.95
Cultura	105	197	1.88

c) Horarios de publicación (interacciones recibidas por publicación según la franja en que se publicó).

Franja	Publicaciones	Interacciones	Interacciones por publicación
Tarde (12-17)	134	289	2.16
Mañana (06-11)	123	247	2.01
Madrugada (00-05)	121	233	1.93
Noche (18-23)	122	231	1.89

d) Impacto de las campañas. Metas promedio por objetivo y engagement realmente observado en las publicaciones asociadas.

Objetivo	Campañas	Meta likes (prom.)	Meta comentarios (prom.)	Meta compartidos (prom.)
Aumentar engagement	165	554.7	266.0	154.7
Captar nuevos usuarios	161	560.4	273.1	159.3
Promover contenido	174	553.4	288.6	173.5

Lectura honesta de los resultados. El dataset es sintético: las diferencias entre categorías (1,88 a 2,12 interacciones por publicación) y entre franjas (1,89 a 2,16) son **pequeñas** y con tan pocos datos no permiten afirmar una tendencia; la franja de tarde (12–17 h) lidera, pero por escaso margen. Además, las interacciones observadas por campaña (máx. 19) son mínimas frente a sus metas (cientos de likes): la mejor alcanza apenas el 6,35 % de su meta (likes + comentarios), lo que refleja el tamaño de la muestra y no necesariamente un

fracaso de las campañas. Los CSV generados en `exportes/` quedan listos para el dashboard de Power BI mencionado en el caso.

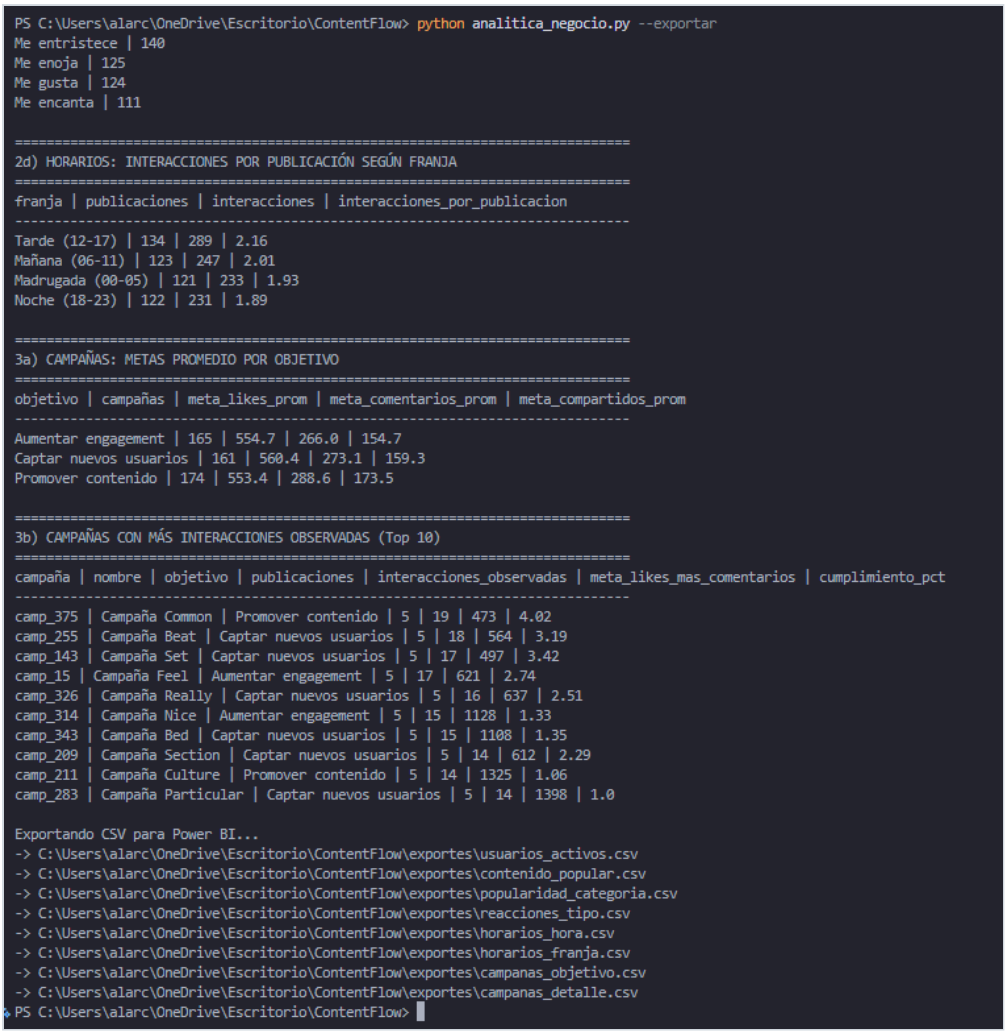


Figura 12. Ejecución de `python analitica_negocio.py --exportar`

Visualización (panel HTML). Con autorización del docente para usar cualquier herramienta de visualización, los resultados se presentan en un panel HTML de un solo archivo (`dashboard/index.html`, sin dependencias externas, con modo claro/oscuro y vista de tabla en cada gráfico). Responde las tres preguntas del caso más el perfil de la audiencia y un resumen de calidad de datos. Se regenera con `python generar_dashboard.py` (lee de Azure; con `--local` usa los JSON).

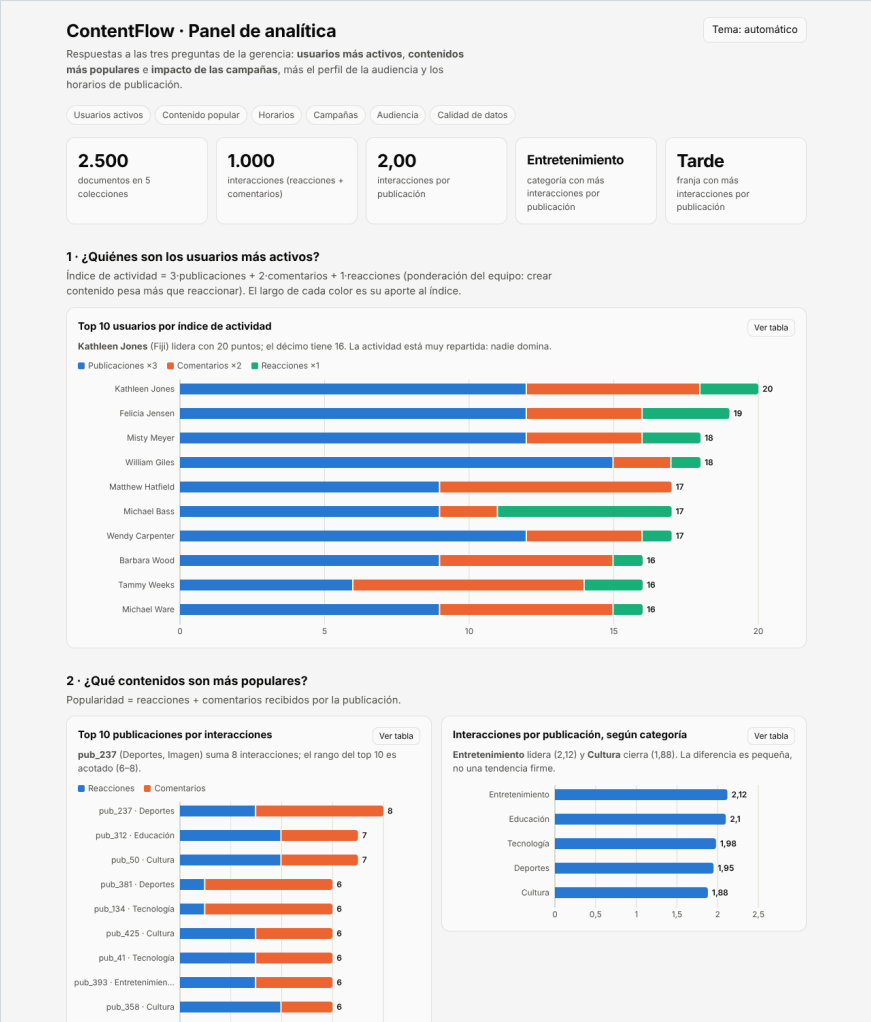


Figura 12b. Panel de analítica de ContentFlow (dashboard/index.html)

4.4 Evaluación de eficiencia y mejoras (IE6)

Metodología. `optimizacion_indices.py` repite cada consulta 5 veces tras calentar la conexión y reporta mediana, bytes devueltos y RU, **antes y después** de cada mejora. Una sola medición no es confiable: incluye establecimiento de conexión y variaciones de red.

Experimento	Hipótesis a comprobar
E1 Proyección	Pedir solo los campos necesarios reduce los **bytes** devueltos sin cambiar los documentos.
E2 `batch_size`	Por defecto PyMongo trae 101 documentos por viaje; con 500 documentos son ≈ 5 viajes a Azure. Un lote mayor reduce viajes y por tanto latencia (posible explicación del 646 ms de la línea base).
E3 Índice `{dispositivo, genero}`	Sin índice Cosmos recorre toda la colección; con índice baja el costo en **RU**.
E4 Índice `{categoria, fecha_publicacion↓}`	Cosmos rechaza ordenar en el servidor sin índice; con él se ordena en el servidor sin traer datos a memoria.
E5 Índice `{objetivo, metricas_objetivo.likes↓}`	Igual que E4 para filtro + orden en campañas.

Experimento	Antes	Después	Variación de tiempo
E1 · Proyección (consulta 1)	documento completo 192.29 ms · 11.168 B · 14.83 RU	solo nombre, correo, país 189.59 ms · 5.090 B · 15.64 RU	+1.4%
E2 · Tamaño de lote al leer 500 campañas	lote por defecto (101) 354.34 ms · 33.593 B · 16.95 RU	batch_size=500 174.52 ms · 33.593 B · 20.66 RU	+50.7%
E3 · Índice compuesto para el filtro de la consulta 1	sin índice (recorre la colección) 189.92 ms · 5.090 B · 15.64 RU	índice {dispositivo, genero} 166.89 ms · 5.090 B · 5.84 RU	+12.1%
E4 · Ordenamiento en servidor (consulta 2)	orden en memoria (el servidor rechaza ordenar sin índice) 198.6 ms · 8.914 B · 15.28 RU	índice {categoria, fecha_publicacion desc} 168.53 ms · 8.914 B · 9.99 RU	+15.1%
E5 · Filtro + orden en campañas (consulta 3)	orden en memoria (servidor rechaza sin índice) 191.74 ms · 9.542 B · 16.44 RU	índice {objetivo, metricas_objetivo.likes desc} 169.06 ms · 9.542 B · 10.67 RU	+11.8%

```
PS C:\Users\alarc\OneDrive\Escritorio\ContentFlow> python optimizacion_indices.py
================================================================================
 EVALUACIÓN DE EFICIENCIA: tiempo (mediana de 5) | volumen | documentos | RU
================================================================================

E1 · Proyección (consulta 1)
  ANTES   (documento completo):   192.29 ms |    11.168 B |  58 docs | 14.83 RU
  DESPUÉS (solo nombre, correo, país):   189.59 ms |     5.090 B |  58 docs | 15.64 RU
  Variación de tiempo: +1.4%   Variación de volumen: -6.078 B

E2 · Tamaño de lote al leer 500 campañas
  ANTES   (lote por defecto (101)):   354.34 ms |    33.593 B | 500 docs | 16.95 RU
  DESPUÉS (batch_size=500):   174.52 ms |    33.593 B | 500 docs | 20.66 RU
  Variación de tiempo: +50.7%   Variación de volumen: +0 B

E3 · Índice compuesto para el filtro de la consulta 1
  ANTES   (sin índice (recorre la colección)):   189.92 ms |     5.090 B |  58 docs | 15.64 RU
  DESPUÉS (índice {dispositivo, genero}):   166.89 ms |     5.090 B |  58 docs | 5.84 RU
  Variación de tiempo: +12.1%   Variación de volumen: +0 B

E4 · Ordenamiento en servidor (consulta 2)
  ANTES   (orden en memoria (el servidor rechaza ordenar sin índice)):    198.6 ms |     8.914 B | 100 docs | 15.28 RU
  DESPUÉS (índice {categoria, fecha_publicacion desc}):   168.53 ms |     8.914 B | 100 docs | 9.99 RU
  Variación de tiempo: +15.1%   Variación de volumen: +0 B

E5 · Filtro + orden en campañas (consulta 3)
  ANTES   (orden en memoria (servidor rechaza sin índice)):   191.74 ms |     9.542 B |  94 docs | 16.44 RU
  DESPUÉS (índice {objetivo, metricas_objetivo.likes desc}):   169.06 ms |     9.542 B |  94 docs | 10.67 RU
  Variación de tiempo: +11.8%   Variación de volumen: +0 B

Resultados guardados en resultados/optimizacion_indices.json
Índices activos: {'usuarios': ['_id_', 'idx_usuarios_dispositivo_genero'], 'publicaciones': ['_id_', 'idx_publicaciones_cat
egoria_fecha'], 'campañas': ['_id_', 'idx_campanas_objetivo_likes']}
```

Figura 13. Ejecución de `python optimizacion_indices.py`

Decisión de ingeniería. `servicios.buscar()` pide el orden al servidor; si Cosmos lo rechaza por falta de índice (`OperationFailure`), lo registra como advertencia y ordena en memoria. Así el sistema funciona con o sin índices y la mejora queda medida y documentada, en lugar de asumir que ordenar en Python es siempre mejor. Para colecciones grandes, ordenar en memoria no escala (hay que traer todos los documentos): ahí los índices son obligatorios.

---

5. Operaciones CRUD completas

5.1 Caso de negocio simulado (`operaciones_crud.py`)

"Lanzamiento de una publicación dentro de una campaña": una persona se registra, publica, recibe reacciones y comentarios, la publicación se asocia a una campaña, se edita y finalmente se retira. Involucra **las 5 colecciones**, no solo una.

Fase	Operaciones	Colecciones
CREATE	`insert_one` de usuario y publicación; 3 reacciones; 2 comentarios; `$addToSet` en la campaña (repetir no duplica)	las 5
READ	`find_one` por `_id`; vista combinada usuario + publicación + conteos; consulta de la campaña que la referencia	las 5
UPDATE	`$set` del contenido y hashtags (los campos inmutables se conservan); cambio de tipo de reacción; 2 likes con `$inc`; `$inc` anidado en `metricas_objetivo`; cambio de dispositivo del usuario	usuarios, publicaciones, reacciones, comentarios, campañas
DELETE	Cascada: reacciones → comentarios → `$pull` en campañas → publicación; luego el usuario de prueba	las 5

Todo lo creado usa ids `*_demo_crud` y se limpia siempre (incluso si algo falla), de modo que el script se repite sin dejar rastro: al final **los conteos y la campaña vuelven exactamente a su estado original**.

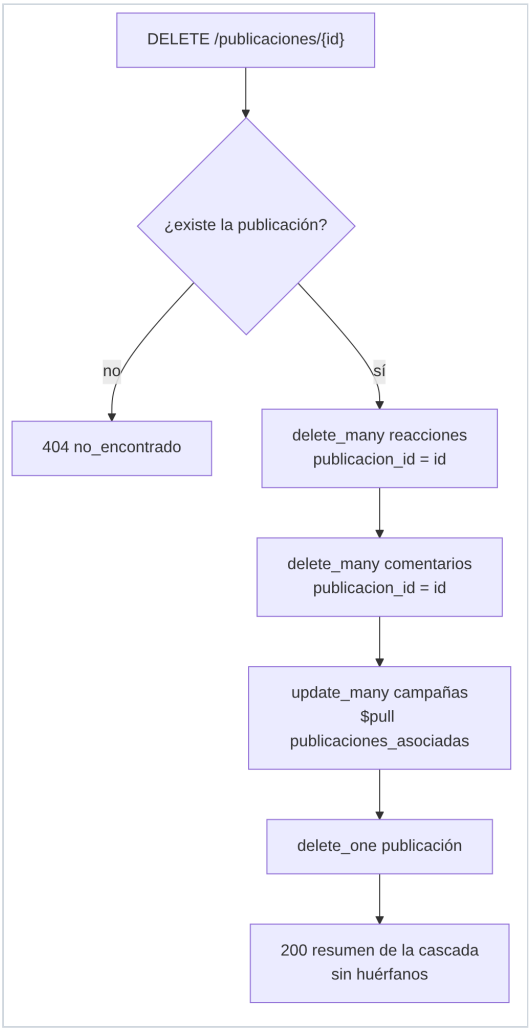


Figura 14. Eliminación en cascada de una publicación

5.2 Validaciones y casos inválidos

El script comprueba que la base y la capa de servicios **rechacen** datos incorrectos (12 casos): categoría inexistente, hashtag mal formado o contenido vacío, publicación de un usuario inexistente, publicación anterior al registro del autor, reacción duplicada, comentario anterior a su publicación o sobre una publicación inexistente, correo ya registrado, `_id` duplicado (`DuplicateKeyError` de la base), modificación de un campo inmutable, actualización de un documento que no existe y borrado de un usuario con actividad. En total son **34 verificaciones**; el script termina con código distinto de 0 si alguna falla.



Figura 15. Resultado de `python operaciones_crud.py` contra Azure (34/34 verificaciones)

5.3 Errores comunes y cómo se tratan (IE8)

Problema	Dónde aparece	Detección	Solución aplicada	Verificación
Duplicación	1 reacción repetida en el dataset; reacción, correo o `_id` repetidos al insertar	`integridad.py`; chequeos en `servicios.py`; `DuplicateKeyError`	Al insertar: se rechaza. En el dataset: se conserva la reacción más antigua y se elimina la otra (`--aplicar`).	Re-auditoría: 0 duplicados
Inconsistencia	48 publicaciones, 245 reacciones y 251 comentarios con fecha anterior a su contexto	`integridad.py`	Al insertar: se rechaza la fecha incoherente. En el dataset: **no se borran** (son datos del cliente), se **marcan** con `revision_temporal = true` para su revisión.	Conteo de marcados = hallazgos

Problema	Dónde aparece	Detección	Solución aplicada	Verificación
Pérdida de contexto	Borrar una publicación dejaría reacciones y comentarios huérfanos	Auditoría de huérfanos; prueba que demuestra el problema con borrado directo	Borrado **en cascada** y limpieza de referencias en campañas; no se permite borrar usuarios con actividad.	0 huérfanos tras el CRUD
Pérdida de actualizaciones	Dos usuarios dando like simultáneamente	—	`$inc` atómico en lugar de leer–sumar–escribir	Likes 0 → 2 en el CRUD
Integridad entre colecciones	Cosmos DB for MongoDB no ofrece transacciones que abarquen varias colecciones (según su documentación)	—	Orden de operaciones seguro (primero dependientes, luego la publicación) e idempotente: si se interrumpe, se puede reintentar sin dejar huérfanos.	Pruebas automáticas

6. Integración con aplicación externa

6.1 Flujo de integración

La aplicación externa es una **API REST** (`api.py`, FastAPI + PyMongo) que expone la base a clientes HTTP como Postman o un frontend. Cada petición se valida, se ejecuta contra Cosmos DB con las mismas reglas de la sección 5 y se responde en JSON.

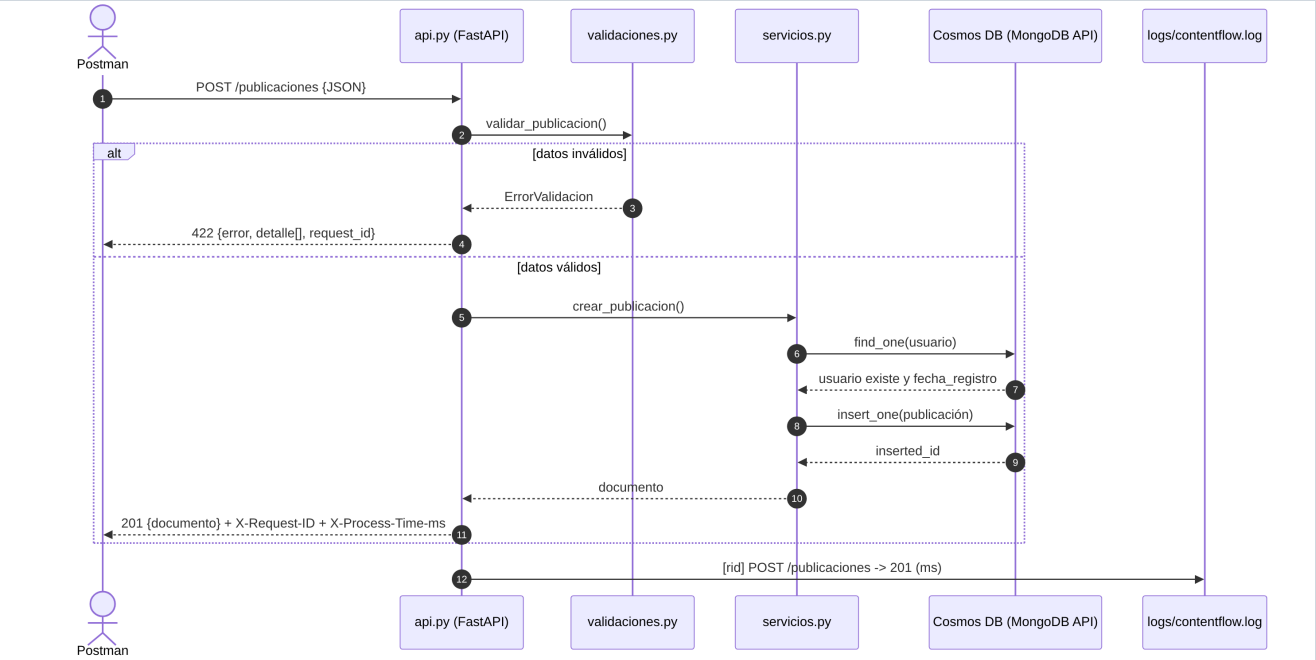


Figura 16. Flujo de una escritura (POST /publicaciones)

Elemento	Detalle
Herramientas	API REST con FastAPI/Uvicorn; cliente Postman; Swagger (`/docs`)
Librerías	`pymongo` (driver), `fastapi`, `uvicorn`, `pandas` (exportación a Power BI) — ver `requirements.txt`
Cadena de conexión	`mongodb://<cuenta>:<clave>@<cuenta>.mongo.cosmos.azure.com:10255/?ssl=true&replicaSet=globaldb&retrywrites=false&maxIdleTimeMS=120000&appName=@<cuenta>@` (clave real solo en `.env`)

Parámetros de la cadena: `ssl=true` cifra el tráfico (TLS 1.2) por el puerto 10255; `replicaSet=globaldb` es el nombre que Cosmos presenta al driver; `retrywrites=false` porque Cosmos DB no admite escrituras reintentables; `maxIdleTimeMS=120000` recicla conexiones inactivas antes de que la red de Azure las cierre.

6.2 Funciones de la API (IE10)

Método y ruta	Función	Operación en la base
`GET /salud`	Disponibilidad y latencia	`ping` + `count_documents`
`GET /publicaciones`	Lista paginada, filtros `categoria`, `tipo`, `usuario_id`	`find` + `sort` + `skip/limit`
`GET /publicaciones/{id}`	Detalle con conteo de reacciones y comentarios	`find_one` + `count_documents`
`POST /publicaciones` · `/usuarios` · `/reacciones` · `/comentarios`	Crear con validación y reglas de integridad	`insert_one`
`PUT /publicaciones/{id}` · `/usuarios/{id}`	Modificar campos permitidos	`update_one` + `$set`
`POST /comentarios/{id}/like`	Like atómico	`find_one_and_update` + `$inc`
`DELETE /publicaciones/{id}`	Eliminar en cascada	`delete_many`, `$pull`, `delete_one`
`POST /campanas/{c}/publicaciones/{p}`	Asociar a campaña	`$addToSet`
`GET /campanas`, `/analitica/*`	Campañas y analítica	`find`, `$group`

6.3 Evidencia de lectura y escritura en tiempo real (IE9)

La colección de Postman (`postman/ContentFlow.postman_collection.json`) ejecuta 19 peticiones con 68 verificaciones: escribe (`POST`), **lee lo escrito** inmediatamente (`GET`, gracias a la consistencia Session), modifica (`PUT`, `$inc`), elimina y confirma el `404`; incluye 6 casos de error controlado. Cada respuesta informa su tiempo en `X-Process-Time-ms`.

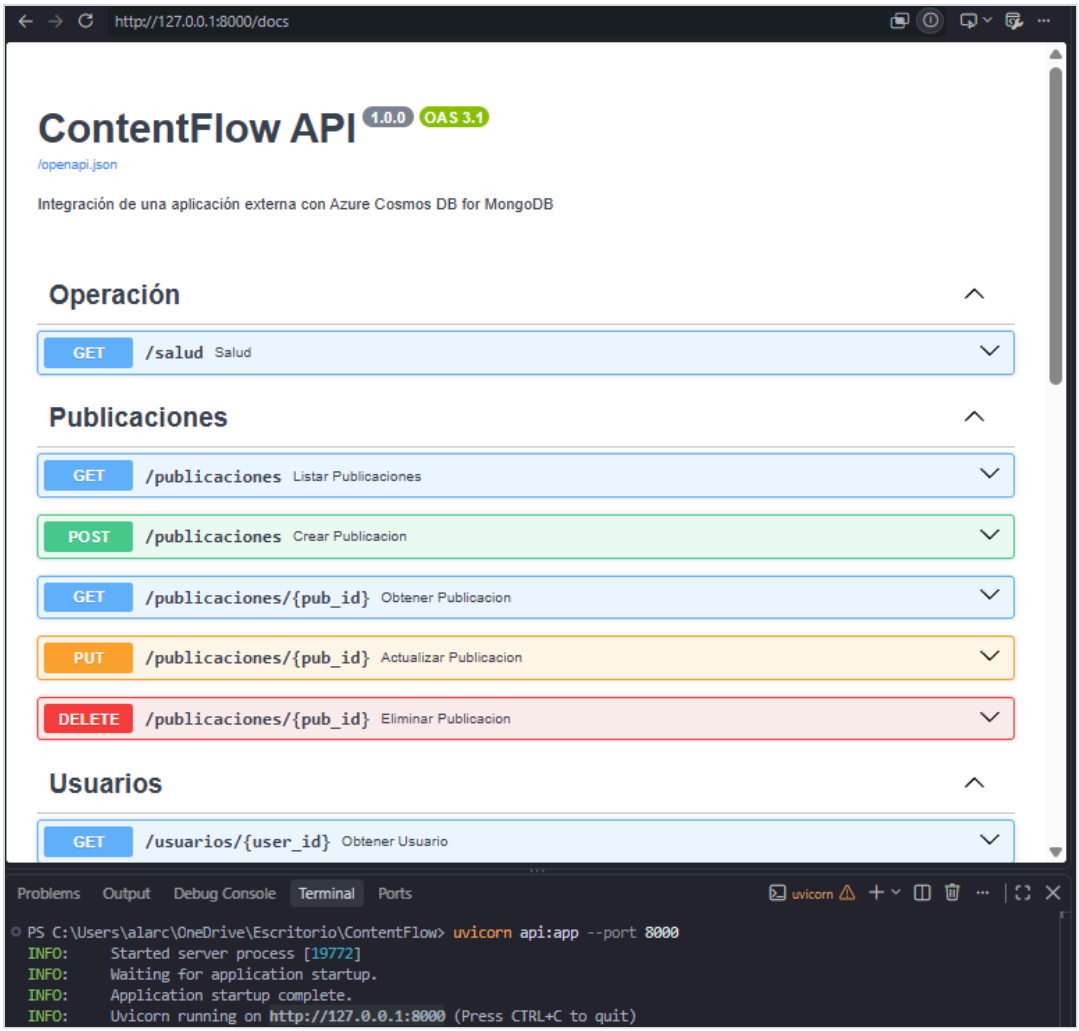


Figura 17. Documentación interactiva (Swagger) de la API



Figura 18. Postman: Run collection contra Azure (todas las verificaciones en verde)

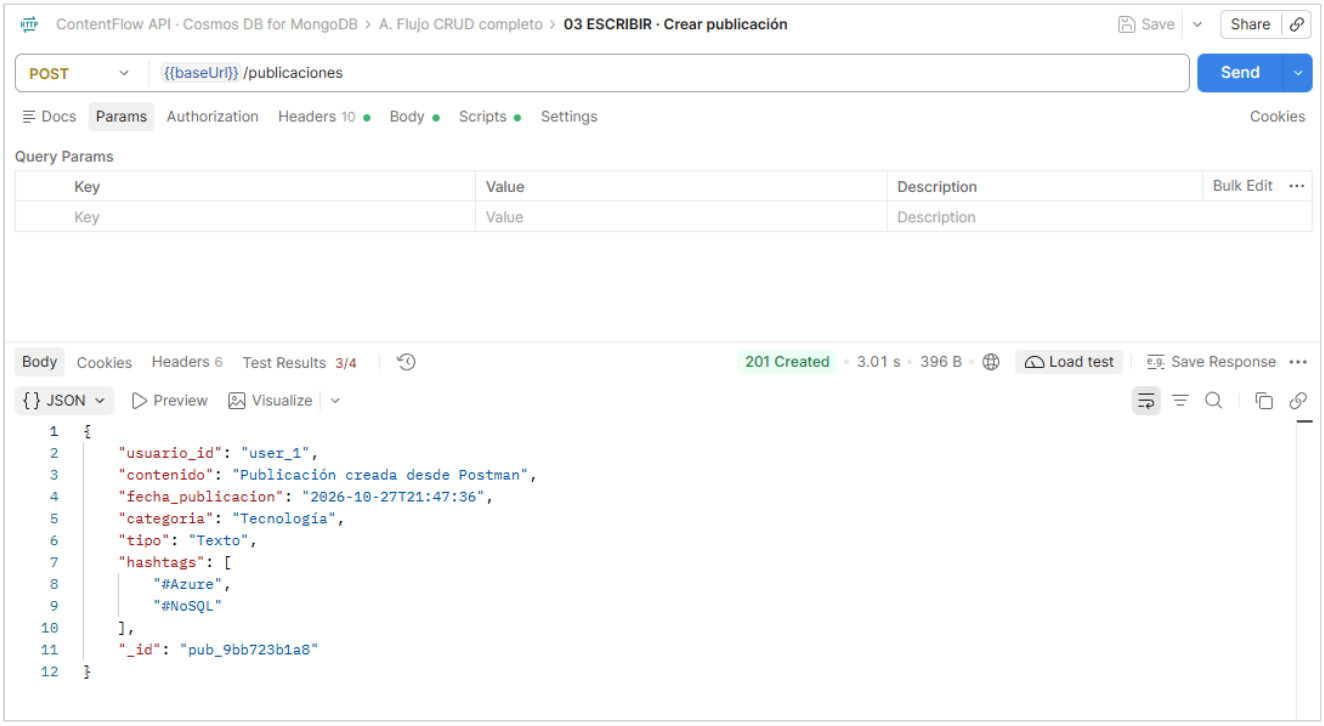


Figura 19. Escritura: POST /publicaciones → 201

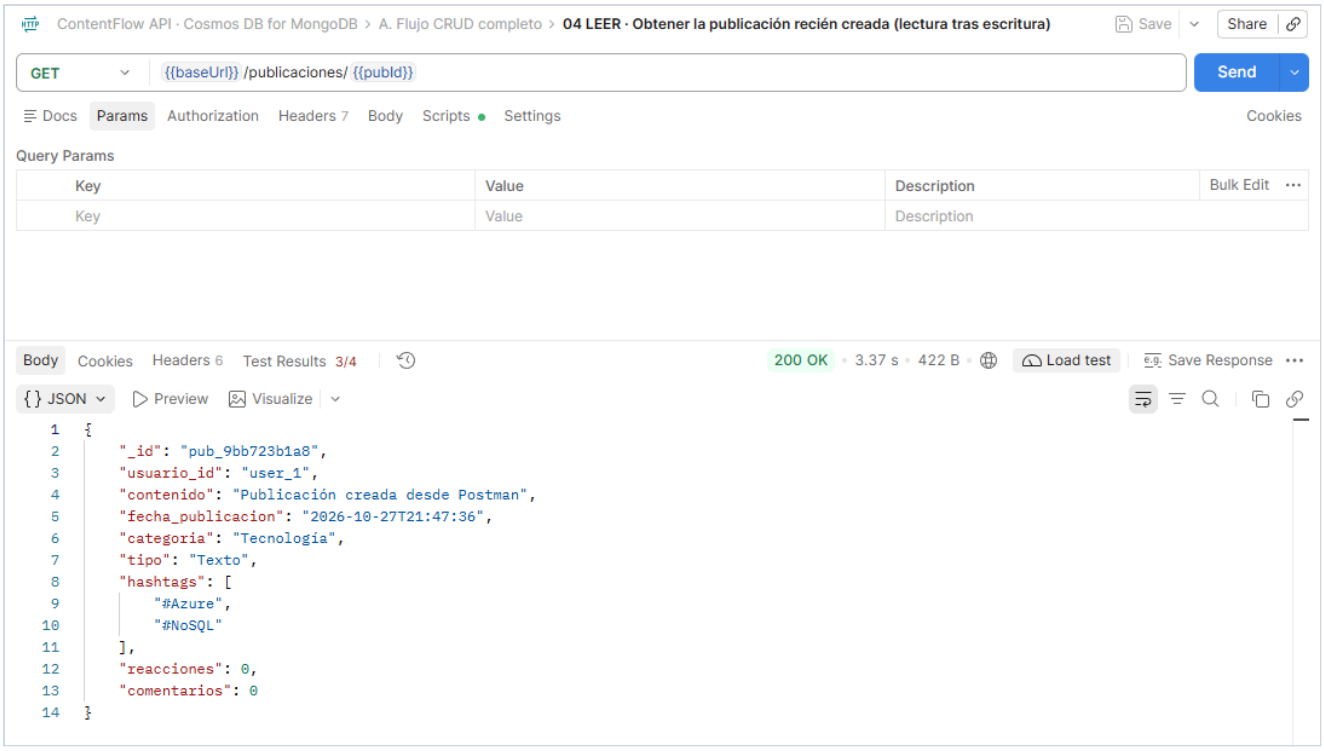


Figura 20. Lectura inmediata de lo escrito: GET /publicaciones/<id> → 200

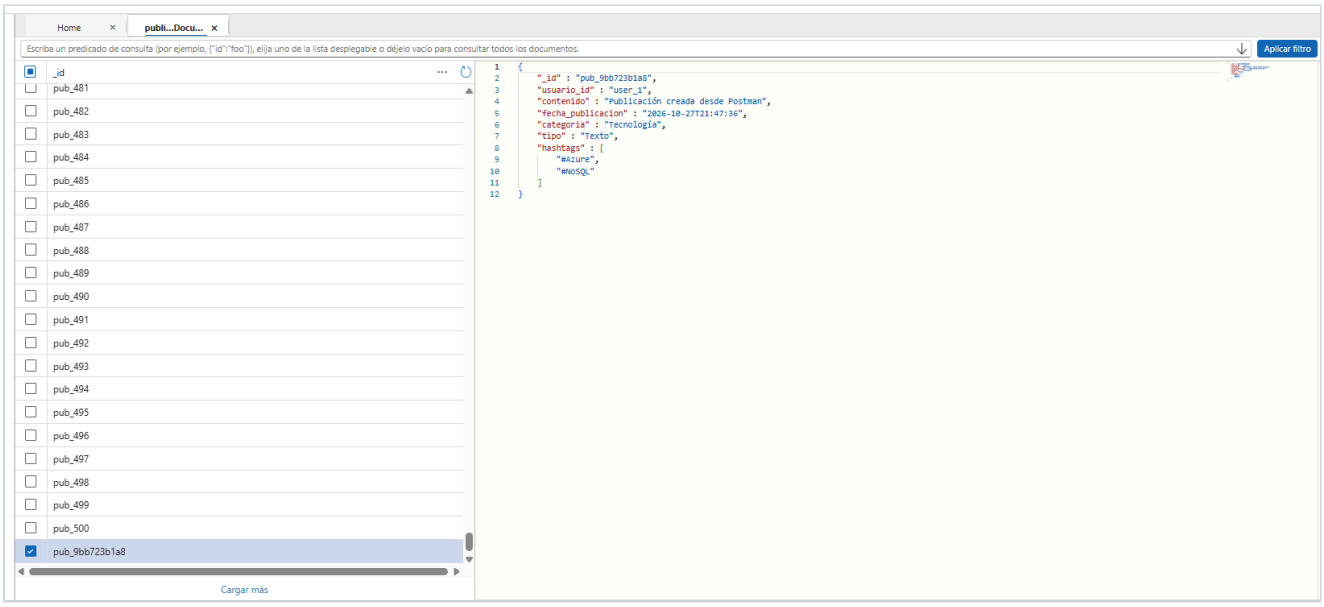


Figura 21. El documento creado por la API visible en Data Explorer (verificación independiente)

---

7. Control de errores y validaciones

7.1 Capas de control

Capa	Mecanismo	Archivo
Conexión	Timeout de 5 s, `ping` previo, mensaje claro si falta/está incompleta la clave (`ConexionError`)	`config.py`

Capa	Mecanismo	Archivo
Validación de entradas	Tipos, longitudes, listas de valores permitidos (categoría, tipo, reacción, dispositivo, género, objetivo), formato de fecha, correo, hashtags (`#palabra`), ids, `likes` entero ≥ 0, **campos desconocidos rechazados**; se informan *todos* los errores a la vez	`validaciones.py`
Reglas de integridad	Existencia de usuario y publicación, coherencia de fechas, sin duplicados, campos inmutables, cascada	`servicios.py`
Excepciones de base de datos	`PyMongoError` / `DuplicateKeyError` / `OperationFailure` capturadas por separado (no `except Exception`)	todos
Respuesta HTTP	`422` validación · `404` no existe · `409` regla de negocio o duplicado · `503` base no disponible · `500` inesperado, siempre con `request_id` y **sin** detalles internos	`api.py`
Trazabilidad	Log rotativo `logs/contentflow.log` (fecha · nivel · módulo · mensaje), `X-Request-ID` por petición, tiempo por petición, credenciales enmascaradas	`config.py`, `api.py`
Pruebas de conectividad	`GET /salud`, `ping` en cada script, casos `503` simulados	`api.py`, `tests/`
Auditoría de datos	Huérfanos, duplicados, fechas incoherentes	`integridad.py`

7.2 Pruebas automáticas

`python -m pytest` ejecuta **51 pruebas** sin necesitar Azure (base simulada con los JSON reales): validadores, las 5 consultas (por ejemplo 58 usuarios y 100 publicaciones de Deportes, coincidentes con lo observado en Azure), las 4 modificaciones, la analítica, el escenario CRUD (34 verificaciones), la auditoría, el respaldo cuando el servidor rechaza ordenar y la API completa (incluido el `503` sin filtrar la clave). Complementan —no reemplazan— las ejecuciones contra Azure.

7.3 Ejemplo de registro de actividad

```
2026-10-27 21:35:54,665 | WARNING  | contentflow.api | [314ab85f6917] regla de negocio: El usuario
'user_999999' no existe (publicación huérfana)
2026-10-27 21:35:54,665 | INFO     | contentflow.api | [314ab85f6917] POST /publicaciones -> 409
(167.7 ms)
2026-10-27 21:35:54,731 | WARNING  | contentflow.api | [0efb9771a430] validación: ["Campos
inmutables, no se pueden modificar: ['usuario_id']"]
2026-10-27 21:35:54,731 | INFO     | contentflow.api | [0efb9771a430] PUT
/publicaciones/pub_d17f394103 -> 422 (1.6 ms)
2026-10-27 21:35:55,335 | WARNING  | contentflow.api | [b7de9a544e0e] regla de negocio: El usuario
ya reaccionó a esta publicación (use actualizar_reaccion)
2026-10-27 21:35:55,336 | INFO     | contentflow.api | [b7de9a544e0e] POST /reacciones -> 409
(528.5 ms)
2026-10-27 21:35:55,579 | INFO     | contentflow.api | [c0168c33acfb] GET
/publicaciones/pub_que_no_existe -> 404 (167.2 ms)
2026-10-27 21:35:55,646 | INFO     | contentflow.api | [4a73835e782d] GET /publicaciones -> 422
(1.0 ms)
2026-10-27 21:35:56,643 | INFO     | contentflow.servicios | publicación pub_d17f394103 eliminada
en cascada: {'publicaciones': 1, 'reacciones': 1, 'comentarios': 1, 'campañas_actualizadas':
2026-10-27 21:35:56,643 | INFO     | contentflow.api | [3c74a58a9a26] DELETE
/publicaciones/pub_d17f394103 -> 200 (906.7 ms)
2026-10-27 21:35:56,904 | INFO     | contentflow.api | [9ece1c0f7133] GET
/publicaciones/pub_d17f394103 -> 404 (174.5 ms)
2026-10-27 21:47:39,954 | INFO     | contentflow.servicios | publicación creada pub_9bb723b1a8
(usuario=user_1)
2026-10-27 21:47:39,956 | INFO     | contentflow.api | [3ec296ae1fb9] POST /publicaciones -> 201
(3003.9 ms)
2026-10-27 21:50:02,208 | INFO     | contentflow.api | [4df81da6cb3a] GET
/publicaciones/pub_9bb723b1a8 -> 200 (3364.1 ms)
```

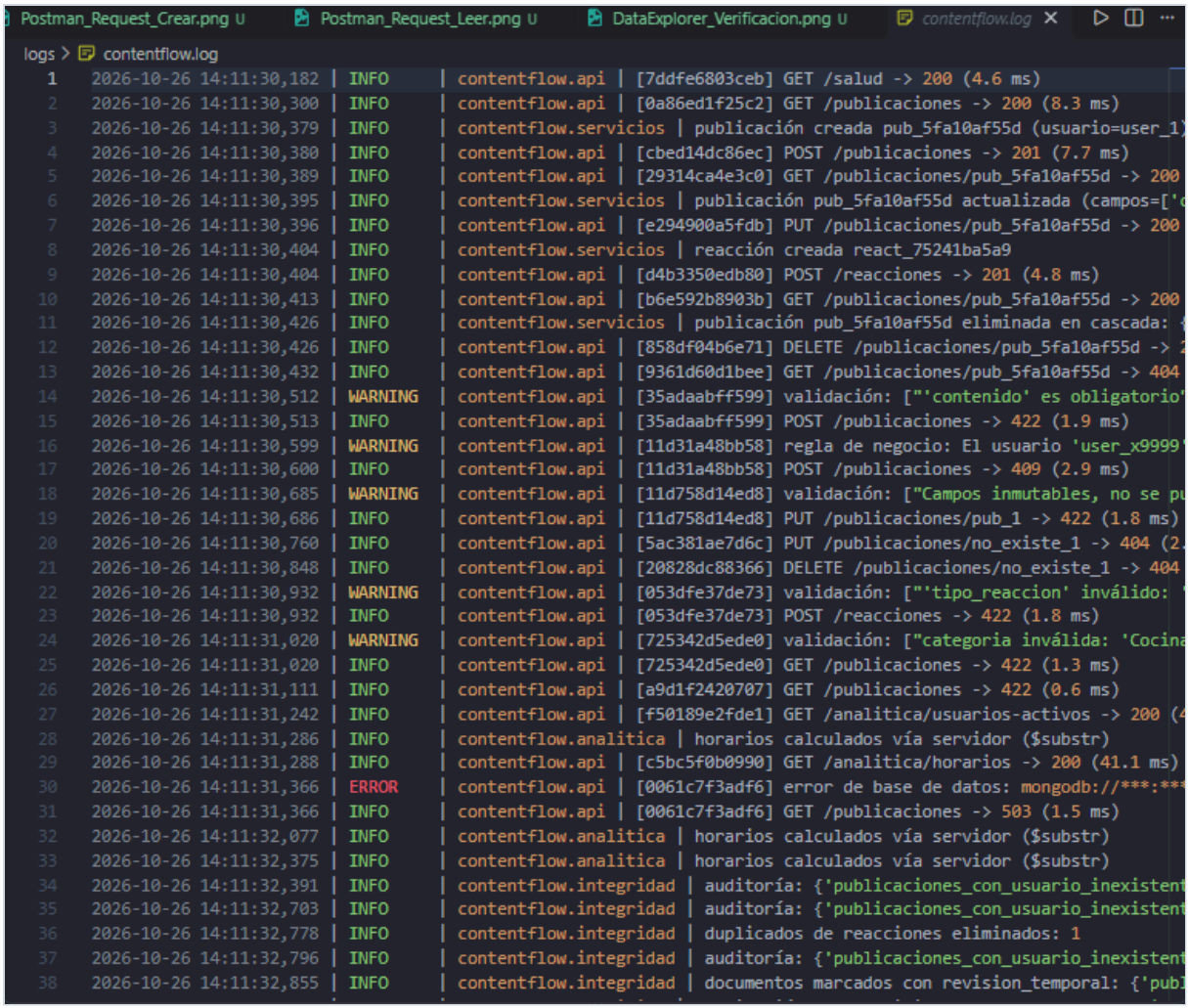


Figura 22. Extracto de logs/contentflow.log tras ejecutar el flujo

---

8. Conclusiones y trabajo futuro

- Se implementó y verificó una base documental en Cosmos DB for MongoDB con 2.500 documentos, consultas de lectura y modificación alineadas con el negocio, un CRUD multi-colección con reglas de integridad y una API REST con trazabilidad.
- **Limitaciones reconocidas:** (1) las colecciones no están particionadas (propuesta en 2.4); (2) el dataset es sintético y pequeño, por lo que las diferencias analíticas son tenues; (3) la integridad referencial depende de la aplicación, por lo que cualquier escritura debe pasar por `servicios.py` — editar directamente en Data Explorer puede reintroducir inconsistencias (la auditoría las detecta).
- **Siguientes pasos:** dashboard en Power BI a partir de los CSV de `exportes/` (el caso lo menciona como objetivo del proyecto); índice único `{usuario_id, publicacion_id}` en `reacciones` una vez eliminado el duplicado existente; restringir la red de la cuenta por IP; rotar claves; para producción, un *job* que mantenga contadores de reacciones precalculados en la publicación en lugar de contarlos en cada lectura.

Anexo A · Cobertura de indicadores

Indicador	Evidencia en este informe
IE1 Implementa la base NoSQL	§2.2, §3
IE2 Parámetros (contenedor, partición, throughput, credenciales)	§2.3–2.7
IE3 Conectividad con herramientas externas	§3.3–3.4, §6
IE4 Consultas de lectura (filtros, proyecciones, orden)	§4.1, §4.3 (y panel HTML)
IE5 Operadores y estructuras según el negocio	§4.1–4.3
IE6 Eficiencia: tiempo y volumen, con mejoras	§4.1, §4.4
IE7 Scripts con múltiples operaciones CRUD (caso de negocio)	§5.1
IE8 Validación: duplicados, inconsistencias, pérdida de contexto	§3.5, §5.2–5.3
IE9 Conexión BD ↔ aplicación en tiempo real	§6.1, §6.3
IE10 Funciones de la aplicación sobre la base	§6.2
IE11 Trazabilidad y control de errores	§7

Anexo B · Reproducción

Ver `README.md`: instalación, orden de ejecución, pruebas y generación de este PDF (`python generar_informe.py`).